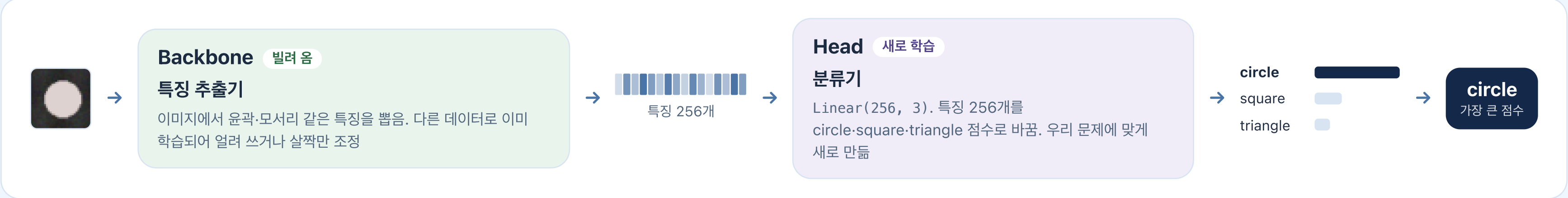
새싹 학습 노트 · Day14 · 2026.11.05 · Vision·멀티모달 AI 응용 · oneday 요약

전이학습 기반 이미지 분류

오늘 한 문장 이미 학습된 모델의 **Backbone(특징 추출기)**을 빌려 와서 우리 문제에 맞는 **Head(분류기)**만 새로 학습하고, 불균형한 데이터에서도 **클래스별로 공정하게** 평가하고, 틀린 이유를 **추정**하는 법을 배웁니다.

1 한 장으로 보는 전이학습

특징 추출은 빌려 오고, 분류만 새로 배운다


Backbone 빌려 옴
특징 추출기
이미지에서 윤곽·모서리 같은 특징을 뽑음. 다른 데이터로 이미 학습되어 얼려 쓰거나 살짝만 조정
특징 256개
Head 새로 학습
분류기
Linear(256, 3). 특징 256개를 circle·square·triangle 점수로 바꿈. 우리 문제에 맞게 새로 만듦
circle
square
triangle
circle
가장 큰 점수

하루 내내 지키는 약속

1. **train**으로만 배우고, **valid**로만 고르고, **test**는 마지막 확인용으로 남긴다
2. 비교할 때는 **한 번에 하나만** 바꾸고, 나머지 조건은 같게 둔다
3. 오답이 없으면 "0건"이라고 쓰고, Grad-CAM은 **원인이 아니라 단서**로 쓴다

2 하루 전체 지도

8교시가 세 묶음으로 이어지고, 끝에서 다음 실험으로 돌아간다

준비 데이터와 모델

- **1교시 데이터셋 설계** "라벨과 분할을 믿을 수 있나?" — ImageFolder · 라벨 계약 · 누수
- **2교시 Backbone·Head** "무엇을 가져오고, 무엇을 새로 만드나?" — requires_grad · BN 모드
- **3교시 Fine-tuning** "어디까지 다시 학습시킬까?" — Head-only · Partial · Full · lr

→ **학습과 평가** 공정하게

- **4교시 클래스 불균형** "적은 클래스를 포기하지 않게 하려면?" — Macro F1 · class weight · Sampler
- **5교시 평가·오류 분석** "어디서 틀렸나? 왜 틀렸을까(가설)?" — Confusion matrix · Precision · Recall

→ **설명과 정리** 믿을 수 있게

- **6교시 Grad-CAM** "모델은 어디에 반응했나?" — target · 설명의 한계
- **7교시 모델 비교·통합** "여러 후보 중 무엇을, 어떤 규칙으로 고르나?" — best epoch · 선택 규칙
- **8교시 재실행·회고** "다른 사람도 같은 결과를 얻나?" — seed · 재현 · 저장 항목

↺ 8교시가 끝나면 **다음 실험 가설** 을 세우고 다시 1교시로 돌아갑니다

3 교시별 핵심

같은 개념이 난이도별로 어떻게 깊어지나

기본필수 동작을 확인한다 → 응용 바꿔 보고 잰다 → 심화 믿어도 되는지 확인한다

교시	기본	응용	심화
1교시 라벨·분할을 믿을 수 있나? ImageFolder · 라벨 계약 · 누수	폴더 → 번호 매핑, 개수 확인	클래스 **비율**, 중복 파일 검사	**그룹 단위** 분할
2교시 무엇을 가져오고 새로 만드나? Backbone/Head · requires_grad · BN 모드	Head 교체, 학습 파라미터 771개	공식 **입력 변환**(Normalize)	**BN 통계** 정책 실험
3교시 어디까지 다시 학습할까? Head-only / Partial / Full · lr	두 전략 accuracy 비교	파라미터 수 · best epoch · 시간	**단계적 해제**, 층별 lr
4교시 적은 클래스를 지키려면? Accuracy의 함정 · Macro F1 · class weight · Sampler	Macro F1 손계산	**Class weight** 적용	**Sampler**와 비교
5교시 어디서·왜 틀렸나? Confusion matrix · Precision/Recall · 오류 이미지	Confusion matrix 읽기	오류 이미지 + 확신도 저장	관찰 → **가설** → 확인
6교시 모델은 어디에 반응했나? Grad-CAM · target · 설명의 한계	정답·오답 1장씩	**target 바꿔** 비교	**Occlusion 실험**
7교시 무엇을 어떻게 고르나? best epoch · 비교표 · 선택 규칙	결과 파일 저장	후보 3개 비교표	**미리 정한 규칙**으로 자동 선택
8교시 같은 결과가 다시 나오나? seed · 재현 · 저장 항목	내가 재실행	짝이 재실행	새 프로세스에서 **복원 검증**

4 코드로 보는 하루 흐름

오늘 배우는 과정을 가장 대표적인 코드 몇 줄로 줄였습니다

- ① **데이터 읽기** (1) `ImageFolder` → 폴더 이름이 라벨 번호가 됨 →
- ② **모델 만들기** (2) Backbone 불러오기 → 얼림 → 새 Head → `eval()` →
- ③ **학습 ×8** (3·4) zero_grad → loss → backward → step → valid loss →
- ④ **best 복원** (7) valid loss가 가장 낮았던 가중치로 되돌림 →
- ⑤ **평가** (4·5) argmax → Macro F1 · confusion matrix →
- ⑥ **설명** (6) target 점수 → gradient → Grad-CAM 열지도 →
- ⑦ **저장** (7·8) 가중치 + 설정 + seed + test 미사용 기록

```python
torch.manual_seed(14)                       # [8교시] seed 고정 → 같은 결과 재현


transform = transforms.Compose([transforms.Resize((32, 32)), transforms.ToTensor()])
train_set = ImageFolder("data/train", transform=transform)
valid_set = ImageFolder("data/valid", transform=transform)
train_set.class_to_idx              # {'circle': 0, 'square': 1, 'triangle': 2}

backbone = load_pretrained_backbone()       # 다른 데이터로 이미 학습된 특징 추출기
for p in backbone.parameters():
    p.requires_grad = False                 # ① Backbone을 얼린다
# [3교시] Partial이면 마지막 Conv 층만 다시 연다: backbone[4].requires_grad_(True)
head = nn.Linear(256, 3)                    # ② 새 Head (특징 256개 → 클래스 3개)
model = nn.Sequential(backbone, head)
backbone.eval()                             # ③ BN 기억값 고정 (얼리기와 별개)

optimizer = torch.optim.AdamW(
    [p for p in model.parameters() if p.requires_grad], lr=0.01)
criterion = nn.CrossEntropyLoss()           # [4교시] 불균형이면 weight=...

best_loss = float("inf")
for epoch in range(8):
    optimizer.zero_grad()                   # 1) 지난 gradient 지우기
    loss = criterion(model(X_train), y_train)  # 2) 예측하고 오차 계산
    loss.backward()                         # 3) gradient 계산
    optimizer.step()                        # 4) 한 걸음 이동
    valid_loss = F.cross_entropy(model(X_valid), y_valid)   # valid는 채점만
    if valid_loss < best_loss:              # [7교시] 가장 좋았던 순간 기억
        best_loss, best_state = valid_loss, copy.deepcopy(model.state_dict())
model.load_state_dict(best_state)

pred = model(X_valid).argmax(dim=1)         # 가장 큰 점수 = 예측
f1_score(y_valid, pred, average="macro")    # 클래스마다 같은 비중으로 평균
confusion_matrix(y_valid, pred)             # 행 = 실제, 열 = 예측

features = conv_layers(image)               # 마지막 Conv 블록의 특징 지도
score = remaining_layers(features)          # 나머지 층 → 클래스 점수 3개
grad = torch.autograd.grad(score[0, target], features)[0]
cam = (grad.mean(dim=(2, 3), keepdim=True) * features).sum(dim=1).relu()

torch.save({"weights": best_state, "classes": train_set.classes,
            "lr": 0.01, "epochs": 8, "seed": 14, "test_used": False}, "model.pt")
```

8교시 seed 고정 무작위 시작값을 맨 앞에서 고정합니다. 그래야 다시 돌려도 같은 결과가 나옵니다.

1교시 ① 데이터 읽기 폴더 이름이 곧 라벨 번호입니다. `class_to_idx`로 매핑을 눈으로 확인하고, 입력 규칙(32×32, 값 0~1)도 여기서 정합니다.

2교시 ② 모델 만들기 얼리고 → 새 Head를 붙이고 → `eval()`. 얼리기는 **가중치**를, `eval()`은 **BN 기억값**을 고정하는 서로 다른 스위치입니다.

3·4교시 ③ 학습 ×8 → ④ best 복원 열린 파라미터만 optimizer에 넘깁니다. 한 epoch = 지우기 → 오차 → gradient → 한 걸음. valid는 채점만 하고, valid loss가 가장 낮았던 가중치를 기억했다가 되돌립니다.

4·5교시 ⑤ 평가 가장 큰 점수가 예측입니다. 클래스마다 같은 비중인 Macro F1과 confusion matrix(행=실제, 열=예측)로 **어디서** 틀렸는지 봅니다.

6교시 ⑥ 설명 · Grad-CAM target 클래스 점수를 마지막 Conv 특징 지도로 미분해 채널별 중요도를 구하고, 점수를 **올린** 곳(`relu`)만 남깁니다.

7·8교시 ⑦ 저장 결과뿐 아니라 **어떻게 만들었는지**(lr·epoch·seed·test 미사용)까지 저장해야 다른 사람이 다시 만들 수 있습니다.

import와 데이터를 텐서로 바꾸는 준비 코드는 생략했고, `load_pretrained_backbone`·`conv_layers` 같은 이름은 이해를 돕기 위해 단순화했습니다.

코드 속 파라미터 · 어디를 고치면 무엇이 달라지나

왼쪽은 위 핵심 코드 그대로, 가운데는 표시한 부분만 고친 코드입니다. 고친 코드로 다시 실행한 결과를 오른쪽에 적었습니다. 위 코드와 실행 조건이 다르면 결과 칸 아래에 조건을 적었습니다. **꼭 기억** 표시가 붙은 7개는 오늘 반드시 가져갈 결과입니다.

숫자·옵션만 바꾸기 줄은 그대로 두고 값만 고쳤습니다

	위 코드 (핵심 코드 그대로)	고친 코드 (이 부분만 바꿔 다시 실행)	결과
3교시 **보폭 lr**	`AdamW(..., lr=0.01)`	`AdamW(..., lr=0.001)`	Accuracy 1.0 → 0.778 8 epoch 안에 덜 배움. 반대로 0.1로 키우면 초반 valid loss가 11.7까지 튐(응용) 기본 데이터
3교시 **epoch**	`for epoch in range(8):`	`for epoch in range(1):`	Accuracy 1.0 → 0.556 triangle을 하나도 찾지 못함 기본 데이터
8교시 **seed**	`torch.manual_seed(14)`	`torch.manual_seed(0)  # 1·42·7도`	Macro F1 0.80 → 0.51~0.58 설정은 같은데 seed만 바꿔도 결과가 이만큼 흔들림 응용 데이터 · 응용 실습 설정(lr 0.003)
꼭 기억	한 번의 결과로 결론 내리지 않는다. seed를 바꿔 여러 번 확인한다		
4교시 **class weight**	`nn.CrossEntropyLoss()`	`nn.CrossEntropyLoss(weight=[0.3, 0.9, 1.8])`	Macro F1 0.804 → 1.0 triangle recall 0.5 → 1.0. 너무 세게(1/개수²) 주면 오히려 0.539 응용 데이터 · 응용 실습 설정(lr 0.003)
꼭 기억	불균형 대응은 "세게"가 아니라 "균형". 적은 클래스를 살리되 과하게 밀지 않는다		
4교시 **평균 방식**	`f1_score(..., average="macro")`	`f1_score(..., average="weighted")`	F1 0.242 → 0.416 모든 이미지를 circle로 예측했는데 점수가 올라감. Accuracy도 0.571로 절반을 넘음 손 예제 (circle 4 · square 2 · triangle 1)
꼭 기억	불균형한 데이터에서 Accuracy·weighted는 실패를 가린다. 클래스마다 같은 비중인 Macro F1을 본다		

코드 한 줄 넣기·빼기 줄을 추가하거나, 지우거나, 주석을 풀었습니다

	위 코드 (핵심 코드 그대로)	고친 코드 (이 부분만 바꿔 다시 실행)	결과
2교시 **사전학습 사용**	`backbone = load_pretrained_backbone()`	`backbone = new_backbone()  # 가중치 안 불러옴`	Macro F1 1.0 → 0.167 모든 이미지를 한 클래스로 예측 기본 데이터
꼭 기억	전이학습의 가치. 특징을 못 뽑는 Backbone 위에서는 Head도 배울 수 없다		
1교시 **입력 규칙**	`Compose([Resize((32, 32)), ToTensor()])`	`Compose([Resize((32, 32)), ToTensor(),` `Normalize(ImageNet 평균, 표준편차)])`	Accuracy 1.0 → 0.933 그런데 평균 확신도는 0.90 → 0.955로 오히려 올라감 응용 데이터
꼭 기억	입력 규칙을 깨면 조용히, 자신 있게 틀린다. 사전학습 때의 입력 규칙을 그대로 지킨다		
2교시 **BN 모드**	`backbone.eval()`	`backbone.train()`	Macro F1 0.956 → 0.790 얼린 층의 BN 기억값이 학습 중에 바뀜 응용 데이터 · Partial 학습
꼭 기억	requires_grad(가중치 고정)와 eval()(BN 기억값 고정)은 서로 다른 스위치다		
3교시 **Partial**	`# backbone[4].requires_grad_(True)`	`backbone[4].requires_grad_(True)  # 주석 해제`	학습 파라미터 771 → 1,923 마지막 Conv 층(1,152개)까지 함께 학습 데이터와 무관
6교시 **양의 반응만**	`cam = (...).sum(dim=1).relu()`	`cam = (...).sum(dim=1)  # relu 뺌`	점수를 낮추는 영역(약 43%)까지 함께 칠해져, 어디가 근거인지 흐려짐 응용 데이터

순서만 바꾸기 같은 줄들의 순서만 바꿨습니다 (흔한 실수)

2교시 **동결 순서**

위 코드 (핵심 코드 그대로):

```python
for p in backbone.parameters():
    p.requires_grad = False
model = nn.Sequential(backbone, head)
```

고친 코드 (이 부분만 바꿔 다시 실행):

```python
model = nn.Sequential(backbone, head)
for p in model.parameters():
    p.requires_grad = False
```

결과: 학습 파라미터 771 → 0
새 Head까지 얼어서 아무것도 배우지 않음. 오류 메시지는 없음
데이터와 무관

꼭 기억 순서 실수는 오류 없이 조용히 일어난다. 학습 파라미터 수를 꼭 출력해 확인한다

모델은 그대로, 고르고 읽는 방법만 바꾸기 학습은 같고, 어떤 모델을 남길지나 무엇을 물을지만 바꿨습니다

	위 코드 (핵심 코드 그대로)	고친 코드 (이 부분만 바꿔 다시 실행)	결과
7교시 **best 복원**	`model.load_state_dict(best_state)`	`# 이 줄을 지움 → 마지막 epoch 그대로 사용`	valid loss 0.006 → 0.277 best 9 epoch 대신 다시 나빠진 16 epoch 모델이 남음 응용 데이터 · lr 0.1, 16 epoch
6교시 **설명할 클래스**	`score[0, target]  # target = 예측 square`	`score[0, 0]  # target = circle`	target 점수 +2.17 → −1.18 같은 square 이미지에 다른 질문을 한 셈. 열지도 모양도 바뀜 응용 데이터

응용·심화는 무엇을 더 요구하나

세 난이도는 같은 코드 흐름을 쓰지만, 요구하는 수준이 다릅니다.

- **기본필수 어떻게 동작하나?** 흐름을 끝까지 돌리고, 각 줄의 의미를 안다 — 결과를 보는 눈 · 제대로 돌아가나
- → **응용 무엇을 바꾸면 좋아지나?** 값 하나를 바꿔 결과 차이를 **재고 설명**한다 — 결과를 보는 눈 · 숫자가 얼마나 달라지나
- → **심화 그 결론을 믿어도 되나?** 그 차이가 **우연·누수·설정 탓이 아님**을 보인다 — 결과를 보는 눈 · 조건을 바꿔도 같은 결론인가

	응용이 더 요구하는 것	심화가 더 요구하는 것
1교시 데이터	클래스 **비율·중복 파일** 검사 같은 파일이 train·valid에 함께 있으면 점수가 부풀려짐	**그룹 단위 분할** 파일은 달라도 같은 원본에서 나온 이미지는 한쪽에만 둠
2교시 모델	사전학습 모델의 **공식 입력 규칙** 지키기 잘못 섞으면 조용히, 자신 있게 틀림	**BN 정책**을 정하고 비교 파라미터를 얼려도 BN 기억값은 따로 바뀔 수 있음
3교시 학습	lr·epoch·전략을 **바꿔 가며 재기** 학습량이 부족하면 Head-only가 덜 배운 채 끝남	**단계적 해제**와 **층별 lr** 새 Head는 크게, 이미 배운 Backbone은 작게 움직임
4교시 불균형	**class weight** 적용 적은 클래스를 되살리되, 너무 세면 오히려 나빠짐(0.539)	**Sampler**와 별도 후보로 비교 둘을 함께 쓰면 보정이 두 번 들어감
5교시 오류	오답을 **확신도와 함께** 저장 자신 있게 틀린 오답이 더 위험	오답마다 **관찰 → 가설 → 다음 확인**을 기록해 다음 실험으로 연결
6교시 설명	**target을 바꿔** 비교 같은 이미지라도 묻는 클래스에 따라 답이 다름	**Occlusion 실험** CAM이 가리킨 곳을 가린 입력에서 점수가 실제로 바뀌는지 확인
7교시 선택	후보 3개를 한 조건씩 바꿔 **비교표**로 나란히	**미리 정한 규칙**(Macro F1 → loss)으로 선택 결과를 본 뒤 기준을 바꾸지 않음
8교시 재현	**짝이 README만 보고 재실행** 내 컴퓨터에서만 되는지 확인	**새 프로세스에서 복원** 저장 파일만으로 예측 확률까지 똑같이 나오는지 확인

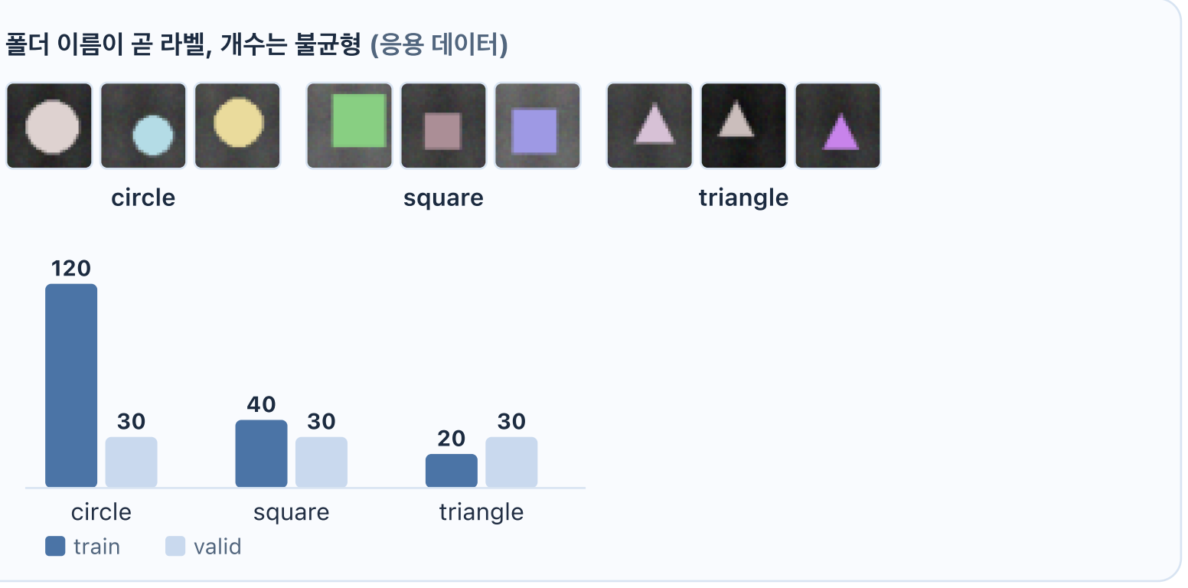
5 전체 파라미터 지도

"무엇을 정하는 값인가"로 묶었습니다. 그림은 헷갈리기 쉬운 개념을 실제 실행 결과로 보여 줍니다

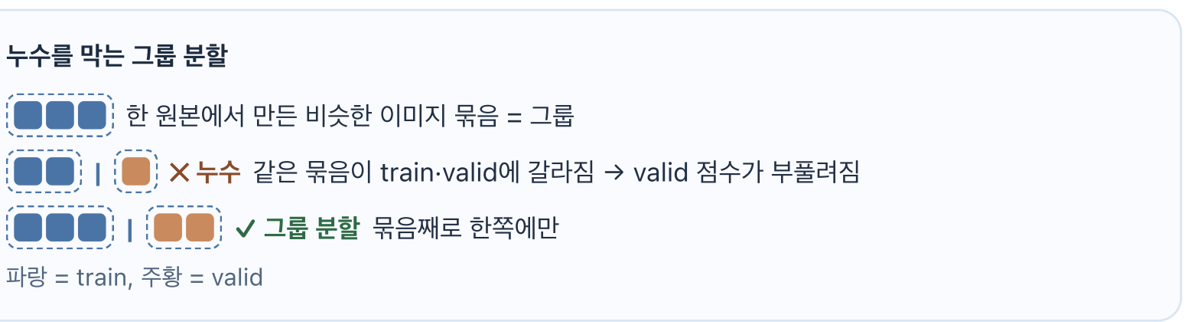
A 데이터와 입력

모델에 무엇을 어떤 형태로 넣나

폴더 이름이 곧 라벨, 개수는 불균형 (응용 데이터)



누수를 막는 그룹 분할


한 원본에서 만든 비슷한 이미지 묶음 = 그룹
✕ 누수 같은 묶음이 train·valid에 갈라짐 → valid 점수가 부풀려짐
✓ 그룹 분할 묶음째로 한쪽에만
파랑 = train, 주황 = valid

폴더 이름 → `class_to_idx`	circle 0, square 1, triangle 2	라벨 계약
train/valid/test 역할	학습/선택/최종 확인	데이터 쓰임 구분
`groups`	group_id	같은 원본은 한쪽에만
`Resize`	32×32	입력 크기 계약
Normalize	없음(작은 모델) / ImageNet 값(ResNet)	가중치와 한 묶음

B 학습 범위

무엇을 바꿀 수 있게 하나

어디까지 열면 몇 개를 배우나 (학습 파라미터 수)

	Conv1	BN1	Conv2	BN2	Head	
Head-only	🔒	🔒	🔒	🔒	열림	771개
Partial	🔒	🔒	열림	🔒	열림	1,923개
Full	열림	열림	열림	열림	열림	2,187개

층별 파라미터 수: Conv1 216 · BN1 16 · Conv2 1,152 · BN2 32 · Head 771

얼리는 스위치는 두 개

- `requires_grad = False` **가중치**를 고정. 학습이 이 숫자를 바꾸지 못함
- `model.eval()` **BN 기억값**(평균·분산)을 고정. 얼려도 `train()`이면 바뀜

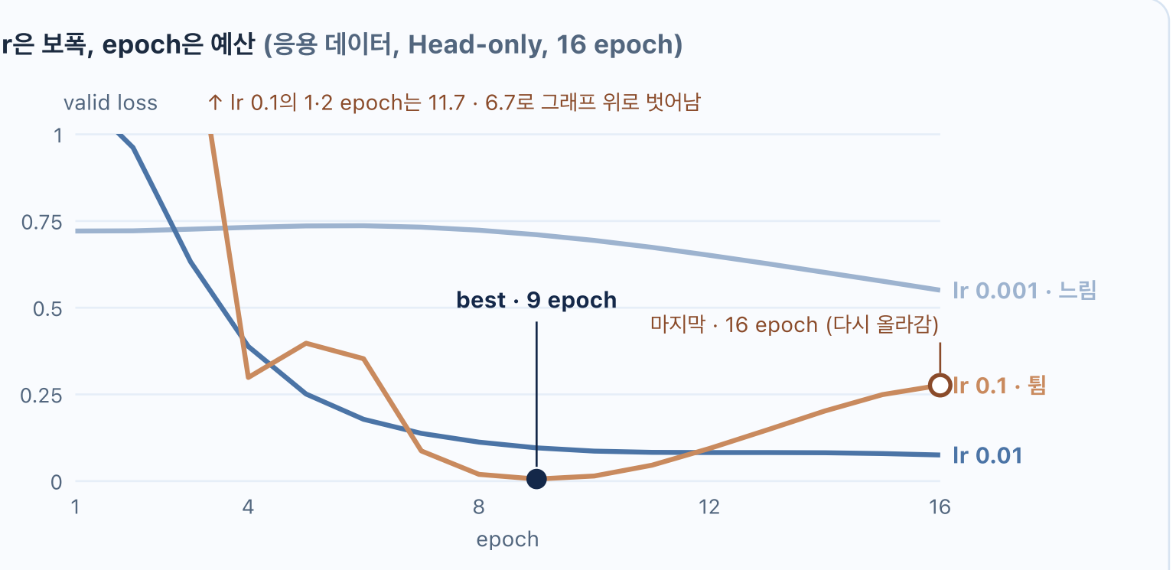
둘은 다른 스위치입니다. Partial에서 BN을 train 모드로 두면 Macro F1 0.956 → 0.790 (응용 데이터)

`Linear(in, out)`	(256, 3)	Head 크기 = 특징 수 × 클래스 수
`requires_grad`	Head만 True	바꿀 숫자 선택
strategy	head / partial / full	여는 범위
Optimizer에 넘기는 목록	열린 것만	실제로 바뀌는 범위
`eval() / freeze_bn`	BN 고정	BN 기억값 고정
`pretrained`	True	사전학습 가중치 사용

C 학습 속도와 양

얼마나 빨리, 얼마나 많이 배우나

lr은 보폭, epoch은 예산 (응용 데이터, Head-only, 16 epoch)



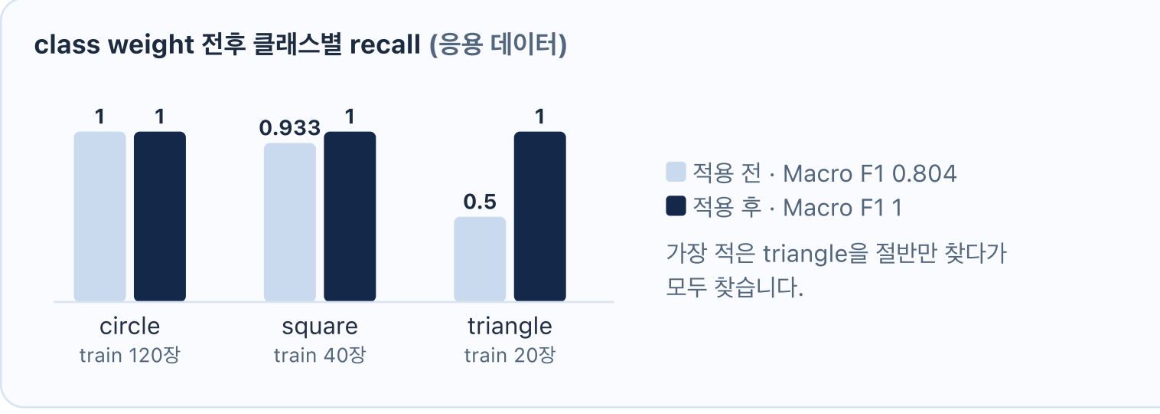
작은 lr은 16 epoch 동안 느리게 내려가고, 큰 lr은 처음에 크게 튑니다. 마지막 epoch가 가장 좋은 epoch는 아니라서 valid loss가 가장 낮았던 순간(best)을 따로 저장합니다.

`lr`	0.003 / 0.01	보폭
층별 lr	Head 0.01, Backbone 0.001	새 층은 크게, 기존 층은 작게
epoch	8 (5·6교시는 1)	학습 예산
`batch_size`	전체 / 16	업데이트 횟수를 바꿈
`switch_epoch`	5	언제 Backbone을 열지
`weight_decay`	0 / 0.0001	가중치 크기 억제
seed	14	무작위 시작값 고정

D 불균형 대응

적은 클래스를 어떻게 지키나

class weight 전후 클래스별 recall (응용 데이터)



가장 적은 triangle을 절반만 찾다가 모두 찾습니다.

두 가지 대응은 다른 방식

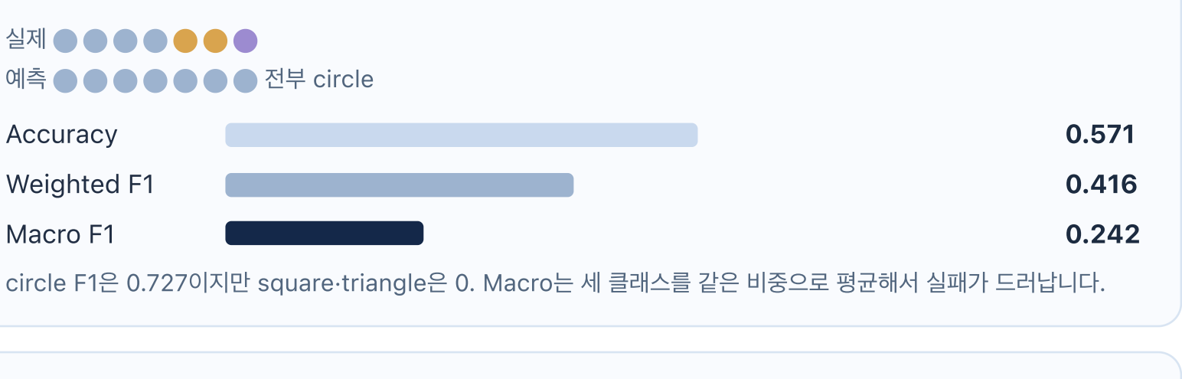
- **class weight = 배점** 뽑는 횟수는 그대로, 적은 클래스 오답에 큰 벌점 — weight = [0.3, 0.9, 1.8] (1/개수, 평균 1)
- **Sampler = 출제 빈도** 벌점은 그대로, 적은 클래스를 더 자주 뽑음 — 둘을 함께 쓰면 보정이 두 번 들어갑니다

class weight	[0.3, 0.9, 1.8]	적은 클래스 오답에 큰 벌점
Weighted Sampler	샘플별 1/개수	적은 클래스를 자주 뽑기

E 평가

어떻게 채점하나

Accuracy는 속인다 · 전부 circle로 예측한 예제



circle F1은 0.727이지만 square·triangle은 0. Macro는 세 클래스를 같은 비중으로 평균해서 실패가 드러납니다.

어디서 틀렸나 · confusion matrix (응용 데이터, class weight 적용 전)

예측 →

↑ 실제	circle	square	triangle
circle	30	0	0
square	2	28	0
triangle	15	0	15

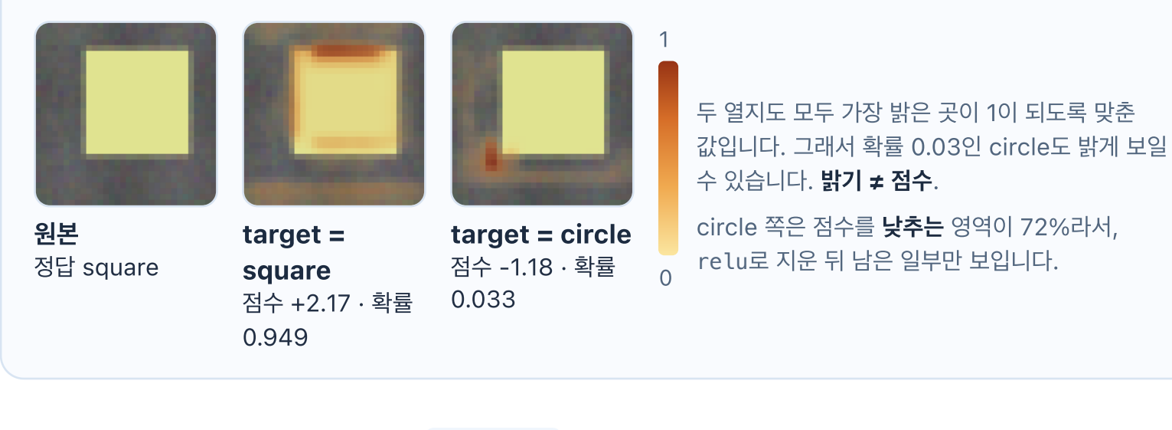
행 = 실제, 열 = 예측. 대각선이 정답입니다. triangle 30장 중 **15장**을 circle로 착각했습니다.

`average`	macro	클래스를 같은 비중으로
Confusion matrix `labels`	[0,1,2]	행=실제, 열=예측 순서
best epoch 기준	valid loss 최소	가장 좋았던 순간 저장
후보 선택 규칙	F1 → loss → 순서	미리 정한 규칙

F 설명

모델이 어디를 봤나

같은 이미지, target만 바꾼 Grad-CAM (응용 데이터)



두 열지도 모두 가장 밝은 곳이 1이 되도록 맞춘 값입니다. 그래서 확률 0.03인 circle도 밝게 보일 수 있습니다. **밝기 ≠ 점수**.

circle 쪽은 점수를 **낮추는** 영역이 72%라서, `relu`로 지운 뒤 남은 일부만 보입니다.

CAM target	예측 클래스	무엇에 대한 설명인가
CAM 대상 층	마지막 Conv 블록	깊이 ↔ 해상도
Occlusion 영역	가장자리 4px	이 부분을 가리고 점수 변화를 확인

출처

NVIDIA AI 코어 엔지니어 2기 · Day14 Vision·멀티모달 AI 응용 · 강의 PDF `Day14_20261105.pdf`「전이학습 기반 Vision 모델 구성과 데이터셋 설계」(99쪽).

실습 코드: Day14 실습 자료 기본필수 01~07, 응용 01~07, 심화 `day14_vision`. 그림과 숫자는 실습 데이터로 직접 실행한 결과입니다(seed 14, CPU 1-thread). 실행 환경에 따라 소수점 아래가 조금 다를 수 있습니다.

오늘 실습의 사전학습 모델은 ImageNet ResNet18이 아니라 합성 도형으로 미리 학습한 작은 CNN입니다. 원리는 같고 크기만 작습니다. 강의 자료의 성능 숫자는 설명용 예시로, 이 문서의 실측 숫자와 맞추는 대상이 아닙니다.